

双项目面试深挖报告

AGI-saber × Mini-Drop | 明日面试速成版

报告定位

这不是项目宣传稿，而是一份以源码、测试和验收材料为准的答辩手册。重点解决三件事：把两条项目主链讲顺；知道每个术语为什么存在；明确哪些结果真的验证过、哪些只能作为设计或下一步。

口径等级	面试中可以怎么说
已实现	源码、配置和合同存在；不自动等于质量提升。
自动化回归通过	证明模块行为和回归门禁符合预期；不自动等于真实环境可用。
受控回放通过	证明固定数据和预算下的策略效果；不等于大量线上真机实验。
Linux live E2E 通过	产生新的 Task、Attempt、Artifact、Evidence 与 Report，才可证明真实链路跑通。

生成日期：2026-09-07 | 适用方向：AI 应用 / 测试开发实习

0. 面试当天先记住这两句话

AGI-saber 总纲

把个人文档变成可检索、可引用的知识，并把普通问答之外的工具任务放进可追踪、可恢复的执行链；重点不是“用了大模型”，而是检索质量、失败降级和评测闭环。

Mini-Drop 总纲

不是让 AI 猜根因，而是让 AI 在受控搜索中选择下一项证据，再由真实采集和 Evidence Gate 决定能否形成结论。

答题固定结构

- 1 先讲业务问题和用户，不从技术栈开场。
- 2 再讲一条端到端链路，让面试官知道系统怎样工作。
- 3 只认领自己能解释源码和取舍的模块。
- 4 最后讲最难的工程问题、验证方法和未完成边界。

最危险的说法

- “全部都是我负责的。” ——范围太大，任何细节都可能被追穿。
- “测试全过，所以效果很好。” ——测试通过、离线质量和线上效果是三件事。
- “模型置信度高，所以结论可信。” ——模型分数、检索分数和真实 Evidence 不能混用。
- “采集失败就说明假设不成立。” ——失败通常只是不可观测，不是科学反证。
- “用了某个框架/模型所以更先进。” ——必须解释需求、取舍、代价和数据证据。

1. AGI-saber: 项目定位与 90 秒回答

1.1 它解决什么问题

- 个人资料散落在 PDF、Markdown 等文件中，关键词搜索难以处理同义表达和上下文。
- 大模型脱离私有资料时容易回答泛化内容，缺少出处，用户无法核对。
- 复杂办公请求可能需要调用工具、多步执行、失败恢复和过程追踪，不能只靠一次 Prompt。
- 系统还需要知道失败发生在路由、召回、融合、重排、生成还是工具执行，而不是只看到最终答案。

1.2 推荐回答

“AGI-saber 是一个基于 Python、FastAPI 和 Vue 的个人办公 Agent，主要解决资料分散、私有知识检索困难，以及模型回答缺少可靠引用的问题。文档上传后会解析、规范化、做父子分块，并建立向量、关键词和可选图谱索引；提问时走查询改写、多路召回、RRF 融合、Rerank、父块回填、去重和无答案门禁，最后生成带引用的回答。知识库问答之外的请求进入统一工具执行链，由 Planner 生成任务和依赖，TaskGraph 校验图结构，GraphRuntime 负责拓扑并行、超时、重试、取消、竞速、快照和有限重规划。我重点负责 Python/FastAPI 后端、RAG 质量链和 Agent Runtime 的实现与验证。当前代码具备完整评测闭环和 274 项通过的自动化回归，但真实业务标注集、RAG 权重和阈值仍需进一步线上验证。”

1.3 个人贡献怎么说

可以认领	不要说满
Python/FastAPI 后端、RAG 质量链、Agent Runtime、评测与全链路联调	“整个前后端、所有数据库和所有模型都由我独立从零完成”
对既有前端 HTTP 合同做 Python 版实现和增强	“所有接口和产品设计都是我原创”
解释具体模块的设计、失败路径、测试和取舍	只背名词却说 “我全部负责”

2. AGI-saber：三条核心链路

2.1 文档入库链

- 1 用户上传 TXT、Markdown 或可提取文本的 PDF。
- 2 PDF 依次尝试 pdftplumber、PyPDF2 和 pdftotext；无法提取有效文字时提示需要 OCR，当前项目本身不执行 OCR。
- 3 文本规范化主要处理换行、空字符、跨行连字符、多余空白和空行；不要声称已经智能删除所有页眉页脚。
- 4 先保存文档和版本，再做 Small-to-Big 切分：父块保留上下文，父块内继续切子块。
- 5 对子块生成向量；子块进入检索投影，父块用于命中后的上下文补全。
- 6 PostgreSQL 或本地 SQLite 保存权威记录；Milvus、Elasticsearch、Neo4j 是可重建的检索投影。

为什么是父子块

父块直接检索时粒度太粗，主题噪声会稀释检索信号；只返回小子块又容易缺上下文。小块负责召回精度，大块负责回答时的语义完整性。当前默认子块 200 字符、重叠 50；父块约 800 字符、重叠 100。这是经验配置，不是已经证明的最优参数。

2.2 RAG 查询链

原始问题 → 按需查询改写（默认 3 个查询）→ Milvus 向量召回 + Elasticsearch BM25 + 可选 Neo4j 图检索 → 单查询或两级 RRF → 远程 Rerank → 命中子块回填父块 → 父块近重复去重 → 合格分数才做无答案判断 → 大模型基于上下文生成 → 返回结构化引用与 Trace。

- RRF 解决的是多路分数尺度不同的问题。它按排名融合，不把余弦、BM25 和图分数直接硬加。
- 当前语义与关键词权重为 0.65/0.35；图检索启用时另有 0.3 权重。参数是配置，不等于已证明最优。
- Rerank 不是再次融合三路，而是对融合后的候选重新判断问题与段落的相关性。
- 0.30 无答案阈值只适用于成功远程 Rerank 后的 0~1 分数；不能拿 RRF 分数直接比较。
- 0.85 是父块字符三元组和编辑相似度的近重复阈值，不是向量余弦阈值。
- 引用是给用户核对原文的信任锚点；Trace 是研发定位召回、融合、重排、生成或降级问题的内部证据链。

2.3 通用执行链

当前顶层主线已经收敛为两类：知识库启用且有内容时走只读 RAG；其他请求进入统一工具执行链。专用“报告 workflow”已移除，报告可以作为普通回答生成，但不会再自动走固定 Research/Writer/Review/Doc 流程。统一执行链内部再根据复杂度选择直接工具或 ReAct 式多步执行。

组件	职责	边界
Router	判断走知识库问答还是统一执行链；执行链内再识别工具/多步需求	不是单独训练的意图分类模型，当前主要是规则与上下文判断
Planner	产出任务节点、参数和依赖关系	计划不是执行结果
TaskGraph	保存节点与边、检查缺失依赖和环、记录节点状态	不负责真正调用工具
GraphRuntime	按拓扑层推进，同层并行，并处理竞速、超时、重试、取消、快照与有限重规划	当前是单机轻量 DAG Runtime，不是分布式调度平台

3. AGI-saber：模型、记忆、安全与降级

3.1 当前模型与外部 API

能力	当前实现	为什么这样选 / 面试边界
聊天与生成	火山方舟 OpenAI 兼容接口; DeepSeek V4 Flash	接入统一、偏低延迟和成本; 仓库没有系统性模型横评, 不能说它已被证明最好
快速任务	fast_model 留空, 实际回退同一个 DeepSeek	查询改写、规划和远程重排目前没有独立小模型
Embedding	豆包多模态 Embedding; 2048 维	文档子块和问题都编码为向量; 当前只传文本, 没有真正做图片 RAG
BGE-M3	未接入	可以作为专用文本向量或本地部署候选, 必须用真实数据比较召回、延迟和成本
联网搜索	可选 Tavily	不属于普通私有知识库 RAG 的必需依赖
Rerank	远程复用 DeepSeek 打分; 本地降级为词法 overlap; Cross-Encoder 仅预留	没有独立 Rerank API; 本地 overlap 不等价于神经重排

3.2 记忆

- 短期记忆：当前保留最近 5 轮原始对话；系统提示词每次另外组装。没有实现“中段历史自动摘要后永久保留”，不要误说。
- 长期记忆：先抽取值得保存的事实或偏好，再保存文本、向量、重要度和类别，运行时按当前问题相关性召回。它不是简单把所有聊天原文塞进 Milvus。
- 长期记忆支持去重、合并、衰减和淘汰；异步抽取意味着最终一致，而不是刚说完就保证所有长期记忆立刻可见。

3.3 工具安全

- 模型只能选择已注册工具；不存在的工具名在执行前拒绝，防止“幻觉工具”。
- 工具调用校验名称和部分参数，统一强参数校验仍有提升空间。
- 命令执行仅在沙箱启用时注册；容器可禁网、只读根文件系统、去权限并限制 CPU、内存、进程数和输出大小。
- 危险命令主要靠规则识别，不是完整 Shell 语法验证；因此应称为纵深防御，不是绝对安全。
- 运行过程记录租户、步骤、耗时、脱敏参数、结果和错误；SSE 给前端进度，Trace 用于审计与排障。

3.4 熔断器

关闭态正常调用远程服务；连续 3 次失败后进入打开态，直接走降级路径，避免所有请求继续等待超时；30 秒后进入半开态，只放 1 个探测请求，成功则恢复关闭，失败则再次打开。这里的 3 次、30 秒和单探针是当前配置，不是通用固定值。

4. AGI-saber：评测、真实证据与简历口径

4.1 RAG 效果怎么评

层次	应该评什么
入库	解析完整性、切块是否破坏关键句/表格、父子映射正确性
召回与排序	Recall@K、MRR、nDCG、Evidence Precision/Recall/F1
生成	答案正确性、Faithfulness、引用支撑度、无答案拒答
工程	P50/P95、错误率、降级率、Token 与 API 成本
消融	纯向量、BM25、二路混合、加图谱、加 Rerank 在同一标注集上对比

4.2 当前已有评测

- 存在 12 条合成脱敏演示用例，覆盖意图、槽位、工具选择、工具参数、RAG 证据、兜底、安全和 Trace。
- 10 项确定性指标：意图准确率、必填槽位召回、工具选择 F1、参数准确率、必须内容召回、RAG 证据 F1、故障恢复、隐私不泄露、边界安全、Trace 完整性。
- Replay 的 baseline 0/12、fixed 12/12 是故意构造的固定夹具，只证明评测闭环能发现错误和验证修复，不代表真实模型从 0% 提升到 100%。
- 本次重新实跑当前源码：274 passed。它证明代码回归通过，不证明 RAG 的真实业务准确率。

4.3 简历数字与当前源码不要混用

项目项	简历历史快照	当前本地源码口径
HTTP 路由	47	README 记录 79 个显式业务路由，其中评测模块 32 个
Alembic 迁移	4	当前目录 10 个迁移版本
Python 测试	122 passed / 4 skipped	本次实跑 274 passed
接口冒烟/压测	9 组 4.355 秒；200 请求、20 并发、166.33 RPS、P95 152.136 ms、0% 错误	属于历史接口证据；不能当 RAG 召回或答案质量指标
专用报告 workflow	简历未单列	当前已移除；保留普通 RAG、统一执行链和独立文档能力

怎么回答版本差异

“简历数字对应投递时的稳定快照；之后继续增加了评测平台、迁移和测试，并移除了不必要的专用报告 workflow。若面试官问当前仓库，我会明确给出当前数字；若问简历指标，我会按当时报告解释。”

5. AGI-saber 高频深挖问答

为什么不用父块直接检索？

父块粒度大，会混入多个主题并稀释信号；子块提高召回精度，命中后回填父块保证上下文完整。

为什么分块需要 overlap？

太小容易把一句话或条件切断；太大会造成索引膨胀、候选重复和重排成本上升。

为什么不能直接把多路分数相加？

余弦、BM25 和图分数的量纲与分布不同；RRF 使用排名融合，减少校准依赖。

为什么无答案门禁放在 Rerank 后？

只有成功远程 Rerank 的 0~1 相关性分数具有当前门禁语义；RRF 分数只是排名函数。

为什么做混合检索？

向量补同义表达，BM25 补专名、编号和精确词，图谱补实体关系和多跳路径；任何一路都不能保证覆盖全部问题。

Retry 和 Replan 的区别？

计划正确但执行偶发失败时重试原节点；发现路线本身不够或工具不合适时，基于已完成结果调整后续图。

为什么没直接上 LangGraph？

当前核心是无环工具任务的依赖调度，自研轻量 DAG 更容易接现有状态与审计；代价是通用持久化、人工审批和复杂循环能力要自己维护，复杂度上升后应重新评估 LangGraph。

最难的工程问题是什么？

让检索链在多依赖失败时仍可用、可定位：主数据与检索投影分离，配合重建、熔断、降级和 Trace；Agent 侧则要保证并发、重试、取消和恢复时不重复产生副作用。

Embedding 为什么不是生成模型？

生成模型优化下一个 token；Embedding 模型输出稳定的固定维向量并针对相似度学习，两者目标和接口不同。

当前最大不足？

真实业务标注集规模小；权重、分块和 0.30 阈值缺少充分消融；多模态模型当前只用文本；本地 overlap 重排质量有限。

6. Mini-Drop: 项目定位与 90 秒回答

6.1 它解决什么问题

- 传统性能诊断依赖工程师 SSH 登录多台 Linux 机器、手工拼命令和整理产物，门槛高且过程难审计。
- 同一现象可能由 CPU、内存、I/O、网络、锁、运行时或下游依赖导致，单链 Agent 容易沿最初猜测一路跑偏。
- 大模型不能直接拥有 Shell、PID 和生产权限，结论也不能只靠模型自信；必须由受控采集和真实 Evidence 约束。

6.2 推荐回答

“Mini-Drop 是面向开发和 SRE 的 Linux 多节点性能诊断平台，解决手工登录机器、跨节点采集、结论依赖个人经验且难以复现的问题。用户描述故障后，Go API 会发现在线 Agent 和新鲜进程，并签发绑定 Agent、PID、启动时间和命名空间的不透明目标句柄。Diagnosis Worker 结合 Knowledge RAG 与已发布 Skill 生成多个可证伪假设，LATS 用 UCT 或显式 PUCT 在预算内选择下一分支。服务端经过工具白名单、运行时能力、权限、预算和审批门禁后创建真实 Task；C++ Agent 执行 perf、py-spy、pprof、eBPF 等采集，产物上传 MinIO，Analyzer 生成火焰图、TopN 和调用图，再由 Evidence Gate 判断支持、反证、中性或受限。Agent 根据真实证据反思、回传价值并继续、转向或停止，最终报告必须引用本次 Evidence。我重点负责 AI 诊断主链的调研、推理编排、证据约束和全链路验收。”

6.3 最难的工程问题

核心答案

难点不是让模型“会分析”，而是把一个不稳定、容易猜答案的模型约束成安全、可恢复、可审计的诊断系统：目标不能被伪造，工具不能越权，失败不能误当反证，产物必须先通过质量校验，重试不能重复产生副作用，最终结论只能引用当前会话的真实 Evidence。

7. Mini-Drop: 完整链路四个核心对象

7.1 从用户请求到最终报告

- 1 React 页面提交现象、服务、环境与故障时间窗；Go API 完成鉴权并创建 Diagnosis。
- 2 服务端读取在线 Agent 和新鲜进程快照，签发短时 opaque binding；模型只能选择签发结果，不能自己编 PID。
- 3 Scope 确认后，Knowledge RAG 提供取证知识，Skill 提供经过验证的路线先验；二者都不是当前事故 Evidence。
- 4 Diagnosis Agent 生成多个可证伪假设；LATS 扩展、评估并选择一个分支。
- 5 Policy、运行时能力、预算和人工审批共同决定能否创建 Tool Call 与 Task。
- 6 C++ Agent 领取 Attempt，白名单 Collector 采集并用短时授权上传 Artifact 到 MinIO。
- 7 Python Analyzer 检查样本质量并产出火焰图、TopN、调用图或指标。
- 8 Artifact 经 Evidence Gate 转成支持、反证、中性或受限 Evidence；采集失败、权限不足等只算不可观测。
- 9 Agent 记录 Reflection，LATS 把有边界的奖励沿路径回传，再决定深入、跨域转向、剪枝或停止。
- 10 满足证据、反证检查、最少轮次和停止条件后生成 VERIFIED 报告；预算耗尽仍无法证明时输出 INSUFFICIENT_EVIDENCE。

7.2 Task、Attempt、Artifact、Evidence

对象	定义	为什么不能合并
Task	用户想完成的一次采集任务	描述目标和期望，不等于已经执行
Attempt	某个 Agent 的一次真实执行	重试要新增 Attempt，不能覆盖第一次失败原因
Artifact	某次 Attempt 产生的原始或派生文件	上传成功只证明文件存在，不证明内容有效
Evidence	Artifact 经解析、质量和主张门禁后形成的结构化事实	只有它才可以支持或反驳报告主张

为什么 Artifact 不能直接当 Evidence

文件可能为空、采错进程、时间窗不匹配、没有正样本、符号缺失或解析失败。系统必须校验来源、hash、目标绑定、样本数、分析状态和主张条件。否则模型会把“有文件”误当成“有事实”。

7.3 状态与终态

- 采集和分析是双状态：COLLECTED 只表示上传完成；Analyzer 成功后才有可用分析结果。
- 分支报告证据不足不等于整个 Diagnosis 立刻结束；只要还有预算和未覆盖证据域，就应继续探索。
- VERIFIED 表示门禁通过且报告引用真实 Evidence；INSUFFICIENT_EVIDENCE 表示在边界内无法证明。会话还可能 FAILED 或 CANCELLED。

8. Mini-Drop: LangGraph、LATS、RAG 与 Skill

机制	负责什么	不负责什么
LangChain create_agent	模型与语义工具循环	不直接执行 Shell，不保存业务真相
LangGraph	线程、节点调度、Checkpoint、可恢复执行	不替代 Mini-Drop 的目标、权限和 Evidence 合同
LATS	多个假设之间的选择、扩展、评估、反思和价值回传	UCT/PUCT 分数不是根因置信度
Knowledge RAG	告诉 Planner 某类现象通常需要采什么证据	命中文档不是本次机器 Evidence
Skill	复用经过评测的调查路线和探针顺序	不复用旧事故答案或旧 Evidence
领域服务	绑定目标、工具白名单、预算、审批、Evidence Gate、持久化和终态	不能把安全边界交给 Prompt

8.1 LATS 的七个页面阶段

选择 → 扩展 → 评估 → 行动 → 观察 → 反思 → 价值回传。论文中的 Simulation 在页面上拆成“行动”和“观察”，算法含义没有改变。

- $UCT = \text{已知平均回报} + \text{对少访问分支的探索奖励}$ ；PUCT 额外引入 Prior。
- Visits、Value、Mean Value、Prior 和 UCT/PUCT 都用于决定下一步查谁，不是 Evidence，也不是报告置信度。
- 实时环境不可回滚，所以 LIVE 只能叫 BUDGETED_LATS：只对选中分支调用真实工具。
- FULL_LATS 只在冻结 Observation Provider 中验证；它可把兄弟 rollout 重置到相同快照，但不会创建真实 Task、Artifact、Evidence 或 Report。

8.2 证据不支持时怎么办

结果	含义	后续
支持	Evidence 支持当前假设	仍做反证检查和交叉验证
反证	Evidence 明确推翻假设	降低奖励、剪枝或转向
中性	数据有效，但既不支持也不反驳	选择信息量更高的新工具
受限	证据有效但质量或覆盖有限	保留限制，补独立证据
不可观测	离线、权限、工具不兼容或执行失败	不能当反证；换工具或在预算内转向

9. Mini-Drop: 安全、可靠性与部署

9.1 安全边界

- PID 会复用，因此目标 binding 同时绑定 Agent、PID、进程启动时间、命名空间和过期时间；模型只看 opaque ID。
- 模型只能选择服务端给出的 allowed tools；工具还要经过目标运行时、Agent capability、内核权限、预算和审批门禁。
- Control 与 Agent 使用 mTLS；Artifact 用短时、限定对象的上传地址，Agent 不持有长期对象存储管理员密钥。
- Go、Python、Java、C++ 专用采集器存在运行时门禁，例如 Go 目标不能误用 py-spy 或 JVM Profile。

9.2 可靠性

机制	解决的问题
幂等键 / effect key	浏览器重试、SSE 重连或 Worker 重启时不重复建任务和事件
租约 + Attempt	Agent 失联后可以由新 Attempt 接管，同时保留历史失败
事务 Outbox	状态入库和事件发布不会出现一个成功、一个丢失
CAS / expected_version	多人或并发干预时拒绝过期树版本覆盖新状态
Checkpoint requested/actual	期望 PostgreSQL 但实际降级内存时必须显示 DEGRADED，不能伪装持久化
SSE + 轮询兜底	实时连接中断时仍可恢复页面状态；领域事件是事实源

9.3 部署

- Control 面使用 Docker Compose 运行 PostgreSQL、MinIO、Control、Go API、Python Worker 和 Web。
- 需要宿主机 PID、perf 与 eBPF 权限的 Worker 可以用 systemd 裸机部署，减少容器权限障碍。
- 云服务器解决计算位置，Docker 解决依赖一致、健康检查、版本切换和回滚；“上云”不等于“不需要容器”。

10. Mini-Drop: 真实验证边界

10.1 当前代码与离线评测

- Python: 447 passed, 3 skipped; Web: 30 个测试文件、126 tests; Go 全包通过; OpenAPI 72 个 method/path 对通过。
- Skill 路线集: 540 个新提示词 Case, 506 个通过; 它测路线召回和安全拒绝, 不是根因准确率。
- 受控根因集: 21 个故障合同生成 540 条确定性回放。关闭 Skill 为 42.78%, 开启为 70.37%。固定 500 组问题同两次工具预算为 41.20% 对 68.00%。
- 这些 540/500 是固定合同与观察的受控回放, 不是几百次公网 Linux 真机注入。

10.2 当前真机验收

能力	状态	能证明什么
Python 源码热点	通过	py-spy 产物、源码函数/行号、四轮调查、Evidence 与报告
Go CPU 热点	通过	pprof 命中真实热点函数; Skill 改变首工具; 四轮证据链
C++ CPU 热点	通过	perf 真实样本命中热点函数; Task 到 Report 链路通过
持续 perf	通过	60 秒任务形成 4 个窗口并生成分析结果
独立 eBPF Campaign	通过	获得非零真实 I/O 样本, 并验证停止清理
Java GC	未完成	已能发现并绑定 JVM, 但完整推进仍出现超时
其余故障场景	仅注入/合同	能启动和停止不等于最终根因诊断逐场景通过

10.3 简历历史指标

简历写的是较早快照: 12 容器、3 个 Agent; 一次白名单 OTel CPU Campaign 中 CPU 0.30% → 96.33% → 0.29% , 输出 SELF_CODE_CPU_HOTSPOT、置信度 0.91 并与 Oracle 一致; 63 项回归; 30 场景 × 3 策略共 90 条记录。面试中只有在能打开对应验收报告、脚本或数据时才逐项认领。当前仓库已经演进到更完整的 LATS、Skill、540/500 评测和多运行时 live E2E, 不能把两个版本的数字拼成同一次实验。

最稳妥的验证回答

“自动化测试证明状态机、接口、解析和页面合同; 540/500 受控回放证明固定预算下的路线策略; Python、Go、C++ 和 perf/eBPF 的 live E2E 才证明真实采集链。Java GC 与其余场景仍未逐项完成, 所以我不会说 21 个场景都诊断通过。”

11. Mini-Drop 高频深挖问答

为什么不让模型直接 SSH?

模型可能编造目标、命令和参数，也难以控制权限、预算和审计。Mini-Drop 让模型只提出语义工具请求，服务端再映射到白名单 Collector。

为什么要绑定启动时间，PID 还不够吗?

Linux PID 会复用。仅凭 PID 可能在进程重启后采到另一个进程，所以 binding 还要包含启动时间、Agent、命名空间和过期时间。

Knowledge RAG、Skill、Evidence 有什么区别?

RAG 给领域知识，Skill 给调查路线先验，Evidence 才是当前机器新采集并通过门禁的事实。前两者不能证明本次根因。

为什么既用 LangGraph 又用 LATS?

LangGraph 管线程、节点调度和 Checkpoint；LATS 管多个假设的搜索和价值回传；领域服务管权限、预算、工具和 Evidence。

为什么 LIVE 不是 FULL_LATS?

真实采集会推进时间并改变系统状态，兄弟分支无法回到完全相同起点；因此只能做预算化渐进搜索。

为什么上传成功还不能生成证据?

文件可能为空、无样本、目标不对或解析失败。必须先做 hash、来源、质量和主张条件检查。

如何防止 AI 一轮猜答案?

设置最少有报告轮次、反证检查、跨域扩展和工具预算；终止由服务端 Evidence 与预算合同决定，不由模型一句“完成”决定。

Skill 如何自进化?

成功报告和反馈生成候选 Skill，经过静态校验、冲突检查、离线评测和人工审批后发布；线上不允许模型自动改 Prompt、代码或权限。

A/B 能证明 Skill 更快吗?

单次顺序运行只能证明路线不同；严格性能结论还需相同目标、窗口和负载、故障重置、随机化、足够样本和统计检验。

失败为什么不是反证?

工具失败只能说明这次没有观察到有效事实；要反证必须有可信 Evidence 明确否定假设。

12. 两个项目如何形成统一叙事

维度	AGI-saber	Mini-Drop
业务目标	个人知识问答与办公工具执行	Linux 多节点性能诊断
核心不确定性	是否召回到正确知识、答案是否忠于引用	哪条根因假设值得继续取证
检索/先验	Milvus + BM25 + 可选图谱, RRF 与 Rerank	Knowledge RAG + Skill 路线检索
真实事实	文档版本、子块/父块与引用	Task、Attempt、Artifact、Evidence
执行编排	自研轻量 TaskGraph / GraphRuntime	LangChain + LangGraph, 叠加领域 LATS
失败处理	熔断、降级、重建索引、Trace	幂等、租约、Outbox、Checkpoint、跨域转向
质量证明	12 条合成评测 + 10 项指标 + 274 项代码回归; 真实标注集不足	540/500 受控回放 + Python/Go/C++ 与 perf/eBPF live E2E
共同主线	把大模型输出转成可追踪、可回归的工程系统	把大模型推理约束在真实证据和安全执行边界内

面试官问：两个项目有什么关系？

“两个项目都不是只封装一次模型调用。AGI-saber 更关注知识检索、工具执行和 Agent 评测；Mini-Drop 更关注真实系统取证、安全目标绑定和证据驱动的根本搜索。前者让我建立了 RAG、Trace、降级和 DAG 调度能力，后者进一步把这些能力放到高风险的性能诊断场景，用 Evidence Gate、LangGraph Checkpoint 和 LATS 约束模型。”

两项目共同的工程亮点

- 模型只负责不确定性决策，事实、权限、状态和副作用由确定性程序控制。
- 都强调可观测：不只返回最终文本，还能定位每个中间阶段。
- 都强调降级与恢复：依赖失败时明确降低能力，而不是悄悄伪装成功。
- 都将“效果好”拆成可复现的数据集、指标、测试和 live 验收。

13. 明天面试前的一小时复习顺序

时间	任务	验收标准
0 ~ 10 分钟	各说一遍两个项目的 90 秒介绍	不看稿；必须包含问题、链路、职责、难点、验证边界
10 ~ 25 分钟	白纸画 AGI-saber 入库链和查询链	能解释父子块、RRF、Rerank、阈值、引用和降级
25 ~ 40 分钟	白纸画 Mini-Drop 从请求到报告	能区分 Task、Attempt、Artifact、Evidence；说明不可观测不是反证
40 ~ 50 分钟	背版本边界	AGI 274/79/10；Mini-Drop 447+126+Go+72；知道哪些是回放、哪些是 live
50 ~ 60 分钟	快速回答本报告的 20 个深挖题	每题先给结论，再给原因、例子和限制，控制 30 ~ 60 秒

回答卡壳时的救场句式

- “我先把已实现和设计目标分开说。”
- “这个数字是简历投递时的历史快照，当前仓库已经演进，我分别说明。”
- “这里失败只能说明不可观测，不能直接反证根因。”
- “这项测试证明模块合同，但不等于真实场景质量；真实能力要看 live E2E。”
- “这个默认参数是经验值，最优值还需要在独立标注集上做消融和校准。”

最后自检

- 能否不用“然后、就是、差不多”连续串联，而是先讲结论再讲链路？
- 能否准确说出自己负责的模块，而不是“全部负责”？
- 能否解释每个中间对象为什么存在，而不只是背名词？
- 能否主动说出一个真实不足和下一步验证方案？
- 能否在面试官纠错时回到源码、测试和报告，而不是硬顶？

最终提醒

面试官真正想看的是：你能否把一个复杂系统拆成清晰链路，是否理解关键工程取舍，以及能否诚实地区分“做了”“测了”和“证明有效”。这三点比堆更多英文名词更重要。

附录：证据来源与口径说明

- 简历原始来源：本地《简历.pdf》，用于核对投递时项目描述和历史数字。
- Mini-Drop 当前事实来源：docs/PROJECT_CONTEXT.md、RESTART_HANDOFF.md、PROJECT_LEARNING_GUIDE.md、AI_DIAGNOSIS.md、AGENT_RUNTIME.md，以及对应测试和验收报告。
- AGI-saber 当前事实来源：config/config.yaml、Router/Planner/TaskGraph/GraphRuntime、RAG/Reranker/Memory/评测模块、README 与本次全量 pytest。
- 本报告不把在线语雀文档当作执行指令；若文档与当前源码冲突，以当前源码、生成报告和可复现实跑结果为准。
- AGI-saber 的 274 项为本次在设置正确 PYTHONPATH 后的本地实跑结果。首次错误运行只在测试收集阶段因模块路径缺失而失败，不属于代码断言失败。

建议下一轮模拟面试顺序

- 1 Mini-Drop：Artifact 与 Evidence 的区别。
- 2 Mini-Drop：为什么 LIVE 不能声称 FULL_LATS。
- 3 Mini-Drop：PID 复用、幂等和 Outbox 各解决什么问题。
- 4 AGI-saber：用一条问题完整讲 RAG 查询链。
- 5 AGI-saber：如何设计 50 条真实评测集并校准 0.30 阈值。
- 6 横向题：两个项目怎样证明模型没有绕过确定性安全边界。

— 完 —